Monoid & Semigroup

Monoid · Fold invoice line totals / log strings · Build with Waqar

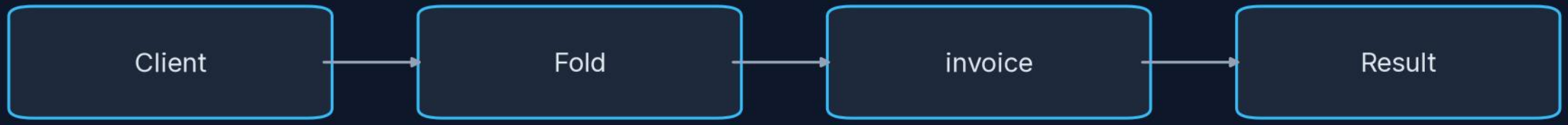
Today's Hook

- Today's force: Fold invoice line totals / log strings
- We earn Monoid by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Monoid, Monoids

The Problem

- Scenario: Fold invoice line totals / log strings
- Constraints: solve «Fold invoice line totals / log strings» with Monoid — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Fold invoice line totals / log strings
- Success looks like: 60

Structure



Monoid: Fold invoice line totals / log strings

Real-World Scenario

- Running example: Fold invoice line totals / log strings
- Key types: Monoid, Monoids
- Happy-path stdout cue: 60
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.monoid.problem;

import java.util.*;

public final class NaiveSum {
  public static int sum(List<Integer> xs){ int s=0; for(int x:xs)s+=x; return s; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Fold invoice line totals / log strings
- Without Monoid, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.monoid.problem;

import java.util.*;
import java.util.function.*;

interface Monoid<A> { A empty(); A combine(A a, A b); }
final class Monoids {
  static final Monoid<Integer> INT_ADD = new Monoid<>(){ public Integer empty(){return 0;} public Integer combine(Integer a,Integer b){return a+b;} };
  static final Monoid<String> STRING = new Monoid<>(){ public String empty(){return "";} public String combine(String a,String b){return a+b;} };
  static <A> A fold(Monoid<A> m, List<A> xs){ A acc=m.empty(); for(A x: xs) acc=m.combine(acc,x); return acc; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.monoid.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Monoid: fold invoice totals ===");
    System.out.println("--- naive ---"); System.out.println(NaiveSum.sum(List.of(10,20,30)));
System.out.println("--- monoid ---");
System.out.println(Monoids.fold(Monoids.INT_ADD, List.of(10,20,30)));
System.out.println(Monoids.fold(Monoids.STRING, List.of("a","b","c")));
  }
}
```

Walkthrough Output

```
=== Monoid: fold invoice totals ===
--- naive ---
60
--- monoid ---
60
abc
```

Step-by-Step Trace

- Match stdout to stages — look for: 60
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 48-monoid-semigroup ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Monoid
// Prefer records + sealed where they shrink the demo.
// Problem under test: Fold invoice line totals / log strings
```

Trade-offs

- Use Monoid when «Fold invoice line totals / log strings» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Monoid in product language.

Related Patterns

- Name the pattern usually confused with Monoid and separate the forces.
- Sketch the roles for «Fold invoice line totals / log strings» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Fold invoice line totals / log strings
- Naive pain → Monoid boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.